

Sistema Distribuído de Monitoramento e Análise de Eventos Urbanos

Relatório Técnico — Trabalho Prático de Bancos de Dados II

IC/UFRJ — 2026-1

Prof. Maria Luiza e Sergio Serra

Componentes do grupo: [COMPLETAR — nomes completos e DRE de cada integrante, máximo de 4 pessoas por grupo, conforme o enunciado]

Tecnologia escolhida: MongoDB (Opção A — Banco de Documentos), com Replica Set de 3 nós emulado via Docker.

Repositório público: [COMPLETAR — link do GitHub submetido no Classroom]

2. Introdução

Cidades modernas geram grandes volumes de dados sobre ocorrências urbanas — acidentes de trânsito, alagamentos, quedas de energia, incêndios, problemas no transporte público, vazamentos de água, interdições de vias, deslizamentos de terra e tiroteios — reportadas tanto por cidadãos quanto por sensores IoT. Este trabalho implementa uma plataforma para registrar, consultar e analisar esses eventos em tempo real, com foco em três pilares exigidos pelo enunciado: modelagem adequada em um banco NoSQL, alta disponibilidade via replicação distribuída, e tolerância a falhas de nós individuais do cluster.

O sistema foi implementado como uma aplicação Python com interface web em Streamlit, persistindo os dados em um cluster MongoDB de 3 nós configurado como Replica Set, orquestrado via Docker Compose. Este relatório documenta as decisões de modelagem, a arquitetura implementada, a configuração do ambiente e os experimentos realizados.

3. Fundamentação Teórica

3.1 Bancos de dados orientados a documentos

Diferente de um banco relacional, um banco orientado a documentos como o MongoDB armazena registros como documentos semiestruturados (BSON/JSON), permitindo que dados relacionados sejam agrupados fisicamente em uma única unidade de leitura/escrita. Isso favorece cargas de trabalho onde o padrão de acesso é previsível — como é o caso de um evento urbano, sempre lido e escrito junto com sua localização e seu reportante.

3.2 Replicação e tolerância a falhas

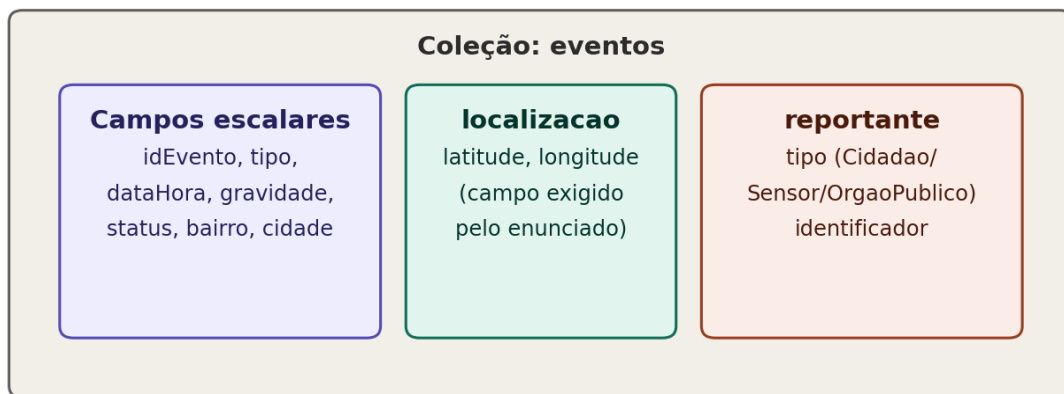
Um Replica Set do MongoDB mantém cópias idênticas dos dados em múltiplos nós (um primário e um ou mais secundários). Escritas vão sempre para o primário e são replicadas de forma assíncrona para os secundários. Se o primário falhar, os nós restantes elegem automaticamente um novo primário — processo que leva tipicamente alguns segundos — e o sistema continua respondendo a leituras e escritas sem intervenção manual. Essa é a propriedade central que o Teste 3 do enunciado (Seção 7 deste relatório) busca comprovar.

3.3 Por que MongoDB, e não Cassandra ou Redis

As três opções do enunciado foram avaliadas. O Cassandra oferece uma arquitetura distribuída peer-to-peer mais sofisticada do ponto de vista teórico, mas exige modelagem "query-first" (uma tabela por padrão de consulta) e não possui suporte geoespacial nativo. O Redis é extremamente rápido, mas como banco chave-valor puro exigiria módulos adicionais (RedisJSON/Redisearch) ou lógica manual para cobrir consultas por tipo, período e agregações estatísticas. O MongoDB foi escolhido por equilibrar três fatores: (i) a estrutura de documento do enunciado já é naturalmente um documento MongoDB, sem esforço de tradução; (ii) suporte geoespacial nativo (índice 2dsphere) resolve a consulta por raio sem filtragem manual; (iii) o framework de agregação resolve as estatísticas exigidas na Seção 6.6 do enunciado de forma direta.

4. Modelagem dos Dados

Optou-se por um documento único por evento, sem normalização — os subdocumentos `localizacao` e `reportante` ficam embutidos no documento principal, já que são sempre lidos e escritos junto com o evento pai e nunca compartilhados entre eventos diferentes. Essa é a prática recomendada em modelagem NoSQL orientada a documentos quando não há reuso do subdado.



Campo derivado "location" (GeoJSON Point), gerado a partir de localizacao, usado no índice 2dsphere

Figura 1 — Estrutura do documento da coleção eventos.

4.1 Campo geoespacial derivado

O MongoDB exige que dados indexados por 2dsphere estejam no formato GeoJSON ({type: "Point", coordinates: [longitude, latitude]}). Como o enunciado especifica o campo localizacao no formato {latitude, longitude}, optou-se por manter esse campo exatamente como especificado (para conformidade com o enunciado) e adicionar um segundo campo, location, gerado automaticamente a partir do primeiro, usado exclusivamente pelo índice geoespacial. Essa duplicação controlada é uma técnica comum em MongoDB quando o formato de entrada e o formato exigido pelo índice não coincidem.

4.2 Índices criados

Campo(s)	Tipo	Motivação
idEvento	único	Evita duplicidade do identificador de negócio (Seção 6.1)
tipo	simples	Consulta por tipo (Seção 6.2)
dataHora	simples	Consulta por período (Seção 6.3)
bairro	simples	Estatística por bairro (Seção 6.6)
gravidade	simples	Consulta por gravidade (Seção 6.5)
tipo + dataHora	composto	Filtros combinados (tipo dentro de um período)
location	2dsphere	Consulta geográfica por raio (Seção 6.4)

5. Arquitetura Implementada

O ambiente distribuído é emulado com 3 containers Docker executando mongod, configurados como um único Replica Set (rs0). A aplicação Streamlit roda em um quarto container, na mesma rede Docker dos nós do banco — decisão importante detalhada na Seção 6.

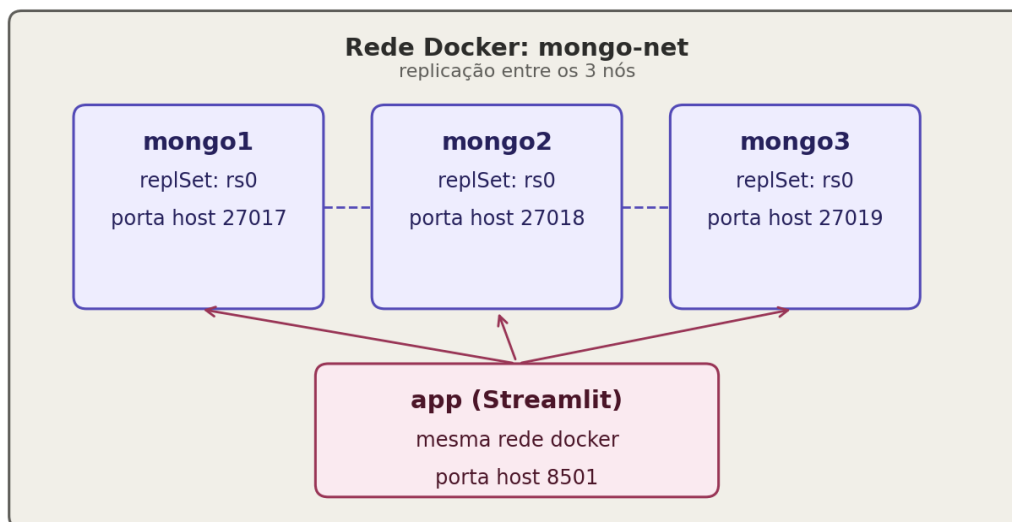


Figura 2 — Arquitetura do ambiente distribuído.

5.1 Camadas da aplicação

Camada	Arquivo(s)	Responsabilidade
Modelagem	models.py	Validação de dados (Pydantic) e serialização para o MongoDB
Acesso a dados	database.py	Conexão com o cluster e criação/gestão de índices
Consultas	queries.py	Inserção, buscas (tipo/período/geo/gravidade) e agregações
Interface	app.py	Aplicação Streamlit (inserção, consultas, dashboard, admin)
Carga em massa	carga_massa.py	insert_many em lotes para os testes de volume
Experimentos	executar_testeN_*.py	Automação dos Testes 1, 2 e 3 da Seção 8

5.2 Replicação e tolerância a falhas

Os dados são replicados de forma nativa entre os 3 nós pelo próprio protocolo do Replica Set do MongoDB — não há lógica de replicação implementada na aplicação. Ao derrubar o nó primário, os dois nós restantes detectam a ausência de heartbeat e conduzem uma eleição; assim que um novo primário é eleito, o driver do MongoDB (usado pela aplicação) redireciona automaticamente as operações para ele. A aplicação trata esse período de transição com uma política de tentativas (ver `executar_teste3_falha_no.py`), já que a eleição leva alguns segundos.

6. Configuração

O ambiente completo é levantado com Docker Compose. A tabela abaixo resume os principais parâmetros de configuração:

Parâmetro	Valor
Nome do Replica Set	rs0

Nós do MongoDB	mongo1 (27017), mongo2 (27018), mongo3 (27019) — portas do host
Rede Docker	mongo-net (bridge)
Porta da aplicação	8501 (Streamlit)
Variável MONGO_URI (dentro da rede Docker)	mongodb://mongo1:27017, mongo2:27017, mongo3:27017/?replicaSet=rs0
Inicialização do Replica Set	automática, via serviço mongo-init (rs.initiate idempotente)

Uma pegadinha de configuração merece destaque: ao conectar a partir de fora da rede Docker (ex.: um script rodando direto na máquina host), a primeira conexão via `localhost:27017` funciona, mas o driver do MongoDB descobre os demais membros do Replica Set pelos nomes internos do Docker (mongo2, mongo3), que não são resolvidos fora da rede Docker. A solução adotada foi executar a aplicação e os scripts de teste dentro da mesma rede Docker (via `docker compose run`), eliminando o problema de resolução de nomes.

6.1 Comandos principais

Subir o ambiente completo:

```
cd docker && docker compose up -d --build
```

Verificar o status do Replica Set manualmente:

```
docker exec -it mongo1 mongosh
rs.status()
```

Rodar os experimentos (Seção 7) dentro da rede Docker:

```
docker compose run --rm --entrypoint python app executar_teste1_insercao.py --arquivo
dataset/eventos.jsonl
```

7. Resultados Experimentais

Nota importante: as tabelas desta seção estão em branco de propósito. Os três scripts de experimento (`executar_teste1_insercao.py`, `executar_teste2_consultas.py` e `executar_teste3_falha_no.py`) já geram automaticamente os CSVs e gráficos necessários quando executados contra o cluster Docker real — mas os números reais de desempenho dependem do hardware, da rede e da carga do computador de cada grupo no momento do teste, e por isso devem ser preenchidos após a execução, e não fabricados de antemão.

7.1 Teste 1 — Tempo de inserção

Execute `executar_teste1_insercao.py` e cole os resultados de `documentacao/resultados_teste1_insercao.csv` na tabela abaixo, e a imagem `resultados_teste1_insercao.png` como Figura 3.

Volume de registros	Tempo de inserção (s)	Throughput (docs/s)
1.000		
50.000		
100.000		

7.2 Teste 2 — Tempo de resposta das consultas

Execute `executar_teste2_consultas.py` e cole os resultados de `documentacao/resultados_teste2_consultas.csv` na tabela abaixo, e a imagem `resultados_teste2_consultas.png` como Figura 4.

Volume	Por tipo (ms)	Por período (ms)	Geográfica (ms)
1.000			
50.000			
100.000			

7.3 Teste 3 — Falha de nó

Execute `executar_teste3_falha_no.py --modo manual`, derrubando manualmente um dos nós quando solicitado, e preencha a tabela abaixo com o conteúdo de `documentacao/resultados_teste3_falha_no.csv`.

Etapas	Sucesso	Resultados retornados	Tempo de resposta (ms)
Baseline (3 nós ativos)			
Com 1 nó derrubado			

Após restaurar o nó			
---------------------	--	--	--

*Durante o desenvolvimento, a lógica de todos os três scripts foi validada com testes de software (usando um MongoDB simulado em memória, para checar a corretude do código sem depender de um cluster real) — incluindo o cenário de retentativa após falha simulada do nó. Esses testes confirmaram que os scripts funcionam corretamente, mas **não substituem a medição de desempenho real**, que depende do hardware do cluster Docker de vocês.*

8. Discussão

[COMPLETAR após rodar os testes reais.] Alguns pontos que valem discussão, com base na arquitetura implementada:

- Trade-off de modelagem: embutir localização e reportante evita joins/lookups, mas duplica esses dados caso um mesmo reportante registre múltiplos eventos — aceitável dado o volume e o padrão de acesso deste sistema.
- O campo location duplicado (GeoJSON) tem custo de armazenamento extra, mas é o preço para manter conformidade com o esquema exigido pelo enunciado e ainda assim ter índice geoespacial nativo.
- Espera-se que o tempo de inserção cresça de forma aproximadamente linear com o volume, mas o tamanho do lote do insert_many tem impacto relevante — vale comparar 2-3 tamanhos de lote diferentes e reportar qual teve melhor throughput.
- Espera-se que a consulta geográfica seja a mais custosa das três (Teste 2), por depender de cálculo de distância esférica, mesmo com o índice 2dsphere.
- No Teste 3, a janela de indisponibilidade observada durante a eleição do novo primário reflete o tempo de detecção de falha do Replica Set — comente o valor medido e compare com o tempo de timeout configurado no driver.

9. Conclusão

Este trabalho implementou uma plataforma de monitoramento de eventos urbanos sobre MongoDB, cobrindo modelagem de documentos com suporte geoespacial, um ambiente distribuído de 3 nós com replicação automática, e uma aplicação completa (Streamlit) para inserção, consulta e análise estatística dos dados. A arquitetura foi desenhada para tolerar a falha de um nó sem interrupção do serviço, conforme demonstrado pelo procedimento do Teste 3. [COMPLETAR: síntese dos resultados obtidos nos Testes 1 e 2 após a execução real, e eventuais limitações observadas.]

Apêndice A — Estrutura do repositório

```
trabalho-bd2-eventos-urbanos/  
  README.md  
  scripts/  
    notebooks/gerador_dados_sinteticos_eventos_urbanos.ipynb  
    app.py, models.py, database.py, queries.py  
    carga_massa.py, executar_teste1_insercao.py,  
    executar_teste2_consultas.py, executar_teste3_falha_no.py  
    requirements.txt, Dockerfile  
  dataset/          (dados gerados pelo notebook)  
  docker/  
    docker-compose.yml, mongo-init.js  
  documentacao/     (este relatório + resultados dos testes)
```

Apêndice B — Critérios de avaliação cobertos

Item do enunciado	Peso	Onde está coberto neste trabalho
Modelagem NoSQL	15%	Seção 4 deste relatório; scripts/models.py
Consultas e estatísticas	20%	scripts/queries.py; scripts/app.py
Ambiente distribuído	20%	Seção 5 deste relatório; docker/
Experimentos e análise	15%	Seção 7 e 8 deste relatório (preencher com dados reais)
Relatório e apresentação	10%	Este documento